

Dynamic Programming: DP on Subsequences

A Problem-Solving Companion to Striver's A2Z DSA Sheet

Maruri Saikiran

June 29, 2026

Contents

1	Dynamic Programming: DP on Subsequences (Knapsack Family)	1
1.1	The Knapsack Family, and Why It All Starts with Subset Sum	1
1.2	Subset Sum Equal to Target	2
1.3	Partition Equal Subset Sum	5
1.4	Count Subsets with Sum K	6
1.5	Count Partitions with a Given Difference	9
1.6	A Note on Assign Cookies: When DP Is the Wrong Tool	11
1.7	Target Sum	12
1.8	0/1 Knapsack	13
1.9	Coin Change (Minimum Coins)	17
1.10	Coin Change II (Count the Ways)	21
1.11	Rod Cutting	24

Chapter 1

Dynamic Programming: DP on Subsequences (Knapsack Family)

Almost every problem in this chapter is the same question asked three different ways: does a subset with property X exist, how many subsets have property X , or what is the best value achievable subject to property X . Settle which of the three you are answering before writing a single line of recurrence.

1.1 The Knapsack Family, and Why It All Starts with Subset Sum

The Recursion chapter's Power Set section's pick/not-pick backtracking tree generates every subsequence of an array in $O(2^n)$. This chapter asks DP's defining question about that same tree: *do many different pick/not-pick paths reach the same (index, target) state*, and if so, can we cache by that state to collapse the exponential tree into a polynomial DP table? For every problem in this chapter, the answer is yes, and the state is almost always *(index, remainingTarget)* – the same two parameters the Recursion chapter's Combination Sum section already recursed on, now with a memo table layered underneath.

The Three Questions, and the 0/1 vs. Unbounded Distinction

- “**Does a subset summing to K exist?**” – a **boolean** DP table, $dp[index][target]$, combining the pick and not-pick branches with a logical OR.
- “**How many subsets sum to K ?**” – the same table, but combining pick and not-pick with **addition** (their counts are disjoint and exhaustive, exactly as in the Recursion chapter's Count Subsequences with Sum K section).
- “**What is the maximum value achievable, subject to a capacity constraint?**” – the same table shape, combining pick and not-pick with **maximum** instead of OR or addition – this is the 0/1 Knapsack problem, the family's namesake.

A second, independent axis determines whether the **pick** branch advances to $index+1$ (each item usable **once** – “0/1” knapsack, as in the Recursion chapter's Subset Sum II section) or stays at $index$ (each item usable **unlimited** times – “unbounded” knapsack, as in the Recursion chapter's Combination Sum section). This chapter covers the 0/1 family first (Sections 1.2–1.8) before the unbounded family (Sections 1.9–1.11).

Every problem here is carried through the same four stages as every earlier DP chapter – recursion, memoization, tabulation, space optimization – with one new wrinkle in the tabulation stage: because the state has *two* dimensions (index and target/capacity), the table is genuinely 2D, and space optimization (when the recurrence only reads the immediately previous index's row) reduces it to a single rolling 1D array, exactly as grid DP's rolling-row trick did in the previous chapter – just with “target/capacity” playing the role “column” played there.

1.2 Subset Sum Equal to Target

Problem Statement. Given an array arr of n non-negative integers and a target k , determine whether **some** subset of arr sums to exactly k .

Pattern Recognition

This is the Recursion chapter's "does a subsequence with sum K exist" recursion, revisited now that we know to look for overlapping subproblems: the state $(index, remaining)$ is reached along many different pick/not-pick paths, so caching by that pair of parameters converts the $O(2^n)$ recursion into an $O(n \cdot k)$ DP.

Deriving the recurrence

Let $f(index, target)$ be **true** if some subset of $arr[0..index]$ sums to exactly $target$. At index $index$, either this element is excluded ($f(index-1, target)$ must hold), or it is included (requiring $arr[index] \leq target$, and $f(index-1, target - arr[index])$ must hold):

$$f(index, target) = f(index-1, target) \vee (arr[index] \leq target \wedge f(index-1, target - arr[index])).$$

Base case: $f(0, target) = (target = 0) \vee (target = arr[0])$.

Approach 1: Recursion

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool subsetSumRecursive(int index, int target, vector<int>& arr) {
5     if (target == 0) return true;
6     if (index == 0) return arr[0] == target;
7
8     bool notPick = subsetSumRecursive(index - 1, target, arr);
9     bool pick = false;
10    if (arr[index] <= target) {
11        pick = subsetSumRecursive(index - 1, target - arr[index], arr);
12    }
13    return notPick || pick;
14 }
15
16 int main() {
17     vector<int> arr = {1, 2, 3, 4};
18     int n = arr.size();
19     cout << (subsetSumRecursive(n - 1, 6, arr) ? "true" : "false") <<
20         endl;
21     return 0;
22 }
```

Approach 1 Complexity

Time. $O(2^n)$. **Space.** $O(n)$ recursion stack.

Approach 2: Memoization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool subsetSumMemo(int index, int target, vector<int>& arr,
5                   vector<vector<int>>& memo) {
6     if (target == 0) return true;
7     if (index == 0) return arr[0] == target;
8     if (memo[index][target] != -1) return memo[index][target];
9
10    bool notPick = subsetSumMemo(index - 1, target, arr, memo);
11    bool pick = false;
12    if (arr[index] <= target) {
13        pick = subsetSumMemo(index - 1, target - arr[index], arr, memo);
14    }
15    return memo[index][target] = (notPick || pick);
16 }
17
18 int main() {
19     vector<int> arr = {1, 2, 3, 4};
20     int n = arr.size(), k = 6;
21     vector<vector<int>> memo(n, vector<int>(k + 1, -1));
22     cout << (subsetSumMemo(n - 1, k, arr, memo) ? "true" : "false") <<
23         endl;
24     return 0;
25 }

```

Approach 2 Complexity

Time. $O(n \cdot k)$. **Space.** $O(n \cdot k)$ memo + $O(n)$ recursion stack.

Approach 3: Tabulation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool subsetSumTabulation(vector<int>& arr, int k) {
5     int n = arr.size();
6     vector<vector<bool>> dp(n, vector<bool>(k + 1, false));
7
8     for (int index = 0; index < n; index++) dp[index][0] = true;
9     if (arr[0] <= k) dp[0][arr[0]] = true;
10
11    for (int index = 1; index < n; index++) {
12        for (int target = 1; target <= k; target++) {
13            bool notPick = dp[index - 1][target];
14            bool pick = false;
15            if (arr[index] <= target) {
16                pick = dp[index - 1][target - arr[index]];
17            }
18            dp[index][target] = notPick || pick;
19        }
20    }
21    return dp[n - 1][k];
22 }
23

```

```

24 int main() {
25     vector<int> arr = {1, 2, 3, 4};
26     cout << (subsetSumTabulation(arr, 6) ? "true" : "false") << endl;
27     return 0;
28 }

```

Dry run of the tabulation table

Dry Run. Take $arr = [1, 2, 3, 4]$, $k = 6$.

$index \backslash target$	0	1	2	3	4	5	6
0 (val 1)	T	T	F	F	F	F	F
1 (val 2)	T	T	T	T	F	F	F
2 (val 3)	T	T	T	T	T	T	T
3 (val 4)	T	T	T	T	T	T	T

At $index = 2$ ($arr[2] = 3$), $target = 6$: not-pick gives $dp[1][6] = F$; pick gives $dp[1][3] = T$ (since $3 \leq 6$). So $dp[2][6] = T$, already confirming a subset exists ($\{2, 4\}$ sums to 6, found even before considering index 3). The final answer, $dp[3][6] = T$, confirms it.

Approach 3 Complexity

Time. $O(n \cdot k)$. **Space.** $O(n \cdot k)$ table, no recursion stack.

Approach 4: Space Optimization

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  bool subsetSumSpaceOptimized(vector<int>& arr, int k) {
5      int n = arr.size();
6      vector<bool> prev(k + 1, false);
7
8      prev[0] = true;
9      if (arr[0] <= k) prev[arr[0]] = true;
10
11     for (int index = 1; index < n; index++) {
12         vector<bool> current(k + 1, false);
13         current[0] = true;
14         for (int target = 1; target <= k; target++) {
15             bool notPick = prev[target];
16             bool pick = (arr[index] <= target) ? prev[target - arr[
17                 index]] : false;
18             current[target] = notPick || pick;
19         }
20         prev = current;
21     }
22     return prev[k];
23 }
24
25 int main() {
26     vector<int> arr = {1, 2, 3, 4};
27     cout << (subsetSumSpaceOptimized(arr, 6) ? "true" : "false") <<
28         endl;
29     return 0;
30 }

```

Approach 4 Complexity

Time. $O(n \cdot k)$. **Space.** $O(k)$ for the rolling row, instead of $O(n \cdot k)$.

Key Takeaway

This boolean-OR recurrence, indexed by $(index, target)$, is the foundation of every 0/1-knapsack-family problem in this chapter – Partition Equal Subset Sum, Count Subsets with Sum K, and Count Partitions with a Given Difference are all this *exact* table, either queried at a derived target or with the OR replaced by addition for counting. Internalize this one table fully before moving on, since nothing new is derived from here through Section 1.5.

1.3 Partition Equal Subset Sum

Problem Statement. Given an array *arr*, determine whether it can be partitioned into **two** subsets with **equal** sum.

Pattern Recognition

“Partition into two equal-sum subsets” reduces directly to Section 1.2: if the whole array can be split into two subsets of equal sum S , then the total sum must be $2S$ (so an **odd** total sum makes this immediately impossible), and finding one subset summing to exactly $S = totalSum/2$ automatically gives the other subset too (everything not in the first subset).

Understanding the reduction

No new recurrence needs deriving: compute *totalSum*; if it is odd, return **false** immediately. Otherwise, ask exactly Section 1.2’s question with target $totalSum/2$. If a subset summing to $totalSum/2$ exists, the remaining elements necessarily sum to the same value (since the two parts must add up to $totalSum = 2 \times totalSum/2$), giving a valid equal partition.

All Four Approaches

Every one of the four stages is Section 1.2’s implementation, called with $k = totalSum/2$. Only the space-optimized version is shown here, to avoid repeating the identical code four times.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool subsetSumSpaceOptimized(vector<int>& arr, int k) {
5     int n = arr.size();
6     vector<bool> prev(k + 1, false);
7
8     prev[0] = true;
9     if (arr[0] <= k) prev[arr[0]] = true;
10
11     for (int index = 1; index < n; index++) {
12         vector<bool> current(k + 1, false);
13         current[0] = true;
14         for (int target = 1; target <= k; target++) {
15             bool notPick = prev[target];
16             bool pick = (arr[index] <= target) ? prev[target - arr[
17                 index]] : false;
18             current[target] = notPick || pick;
19         }
20     }
21 }
```

```

19     prev = current;
20 }
21 return prev[k];
22 }
23
24 bool canPartition(vector<int>& arr) {
25     int totalSum = accumulate(arr.begin(), arr.end(), 0);
26     if (totalSum % 2 != 0) return false;
27
28     return subsetSumSpaceOptimized(arr, totalSum / 2);
29 }
30
31 int main() {
32     vector<int> arr = {1, 5, 11, 5};
33     cout << (canPartition(arr) ? "true" : "false") << endl; // true
34     return 0;
35 }

```

Dry run

Dry Run. Take $arr = [1, 5, 11, 5]$.

$totalSum = 22$, even, so target = 11. Running Section 1.2's subset-sum check for target 11: the subset $\{5, 5, 1\}$ sums to 11 – found, so the answer is **true**. The complementary subset $\{11\}$ also sums to 11, confirming the equal partition $\{1, 5, 5\}$ and $\{11\}$.

Complexity Analysis

Time Complexity. $O(n \cdot totalSum)$, inherited directly from Section 1.2's subset-sum DP with $k = totalSum/2$. **Space Complexity.** $O(totalSum)$ for the rolling row (space-optimized version).

Key Takeaway

Before deriving a new DP recurrence, always check whether the problem is a thin wrapper around one already solved – here, an odd-sum early exit plus a single call to subset-sum with a derived target. This is the same “reduce to an already-solved sub-problem” instinct already exercised for House Robber II's circular-array reduction back in the DP 1D chapter.

1.4 Count Subsets with Sum K

Problem Statement. Given an array arr of non-negative integers and a target k , count **how many** subsets sum to exactly k .

Pattern Recognition

This is Section 1.2's exact recurrence with the logical OR replaced by **addition** – the same exists-to-count transformation as the Recursion chapter's “Check if There Exists a Subsequence with Sum K” to “Count Subsequences with Sum K” transition, now applied on top of the memoized/tabulated table instead of plain recursion.

Deriving the recurrence

$$f(index, target) = f(index-1, target) + (arr[index] \leq target ? f(index-1, target-arr[index]) : 0).$$

Base case (handling a possible zero-valued element carefully, since a 0 can be included or excluded without changing the sum, doubling the count): if $index = 0$, return 2 if $target = 0$ and $arr[0] = 0$; return 1 if $target = 0$ or $target = arr[0]$ (but not both conditions above); else return 0.

Approach 1: Recursion

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int countSubsetsRecursive(int index, int target, vector<int>& arr) {
5     if (index == 0) {
6         if (target == 0 && arr[0] == 0) return 2;
7         if (target == 0 || target == arr[0]) return 1;
8         return 0;
9     }
10
11     int notPick = countSubsetsRecursive(index - 1, target, arr);
12     int pick = 0;
13     if (arr[index] <= target) {
14         pick = countSubsetsRecursive(index - 1, target - arr[index],
15                                     arr);
16     }
17     return notPick + pick;
18 }
19
20 int main() {
21     vector<int> arr = {1, 2, 2, 3};
22     int n = arr.size();
23     cout << countSubsetsRecursive(n - 1, 3, arr) << endl; // 3
24     return 0;
25 }

```

Approach 1 Complexity

Time. $O(2^n)$. **Space.** $O(n)$ recursion stack.

Approach 2: Memoization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int countSubsetsMemo(int index, int target, vector<int>& arr,
5                     vector<vector<int>>& memo) {
6     if (index == 0) {
7         if (target == 0 && arr[0] == 0) return 2;
8         if (target == 0 || target == arr[0]) return 1;
9         return 0;
10    }
11    if (memo[index][target] != -1) return memo[index][target];
12
13    int notPick = countSubsetsMemo(index - 1, target, arr, memo);
14    int pick = 0;
15    if (arr[index] <= target) {
16        pick = countSubsetsMemo(index - 1, target - arr[index], arr,
17                                memo);
18    }
19    return notPick + pick;
20 }

```

```

17     }
18     return memo[index][target] = notPick + pick;
19 }
20
21 int main() {
22     vector<int> arr = {1, 2, 2, 3};
23     int n = arr.size(), k = 3;
24     vector<vector<int>> memo(n, vector<int>(k + 1, -1));
25     cout << countSubsetsMemo(n - 1, k, arr, memo) << endl; // 3
26     return 0;
27 }

```

Approach 2 Complexity

Time. $O(n \cdot k)$. **Space.** $O(n \cdot k)$ memo + $O(n)$ recursion stack.

Approach 3: Tabulation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int countSubsetsTabulation(vector<int>& arr, int k) {
5     int n = arr.size();
6     vector<vector<int>> dp(n, vector<int>(k + 1, 0));
7
8     dp[0][0] = (arr[0] == 0) ? 2 : 1;
9     if (arr[0] != 0 && arr[0] <= k) dp[0][arr[0]] = 1;
10
11     for (int index = 1; index < n; index++) {
12         for (int target = 0; target <= k; target++) {
13             int notPick = dp[index - 1][target];
14             int pick = (arr[index] <= target) ? dp[index - 1][target -
15                 arr[index]] : 0;
16             dp[index][target] = notPick + pick;
17         }
18     }
19     return dp[n - 1][k];
20 }
21
22 int main() {
23     vector<int> arr = {1, 2, 2, 3};
24     cout << countSubsetsTabulation(arr, 3) << endl; // 3
25     return 0;
26 }

```

Dry run of the tabulation table

Dry Run. Take $arr = [1, 2, 2, 3]$, $k = 3$.

$index \backslash target$	0	1	2	3
0 (val 1)	1	1	0	0
1 (val 2)	1	1	1	1
2 (val 2)	1	1	2	2
3 (val 3)	1	1	2	3

The three qualifying subsets summing to 3 are $\{1, 2\}$ (using the first 2), $\{1, 2\}$ (using the second 2), and $\{3\}$ alone – matching $dp[3][3] = 3$.

Approach 3 Complexity

Time. $O(n \cdot k)$. **Space.** $O(n \cdot k)$ table, no recursion stack.

Approach 4: Space Optimization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int countSubsetsSpaceOptimized(vector<int>& arr, int k) {
5     int n = arr.size();
6     vector<int> prev(k + 1, 0);
7
8     prev[0] = (arr[0] == 0) ? 2 : 1;
9     if (arr[0] != 0 && arr[0] <= k) prev[arr[0]] = 1;
10
11     for (int index = 1; index < n; index++) {
12         vector<int> current(k + 1, 0);
13         for (int target = 0; target <= k; target++) {
14             int notPick = prev[target];
15             int pick = (arr[index] <= target) ? prev[target - arr[index]] : 0;
16             current[target] = notPick + pick;
17         }
18         prev = current;
19     }
20     return prev[k];
21 }
22
23 int main() {
24     vector<int> arr = {1, 2, 2, 3};
25     cout << countSubsetsSpaceOptimized(arr, 3) << endl; // 3
26     return 0;
27 }

```

Approach 4 Complexity

Time. $O(n \cdot k)$. **Space.** $O(k)$ for the rolling row.

Key Takeaway

Counting subsets and checking subset existence differ by exactly one operator (addition instead of logical OR) at every step of an otherwise identical recurrence – the same lesson already drawn from the Recursion chapter’s exists-to-count transition. Watch for the zero-value edge case specifically whenever a problem allows zero-valued array elements, since a 0 can always be freely included or excluded without affecting the sum, silently doubling some counts if not handled in the base case.

1.5 Count Partitions with a Given Difference

Problem Statement. Given an array arr and an integer d , count the number of ways to partition arr into two subsets S_1 and S_2 such that $S_1 - S_2 = d$ (where S_1, S_2 denote the sums

of the two subsets, and every element belongs to exactly one of them).

Pattern Recognition

Another reduction to an already-solved problem: given $S_1 - S_2 = d$ and $S_1 + S_2 = totalSum$ (every element is in exactly one subset), solving this small system of two equations gives $S_1 = (totalSum + d)/2$ – turning the partition-counting question into exactly Section 1.4’s “count subsets summing to K ,” with $K = (totalSum + d)/2$.

Understanding the reduction

Adding the two equations $S_1 - S_2 = d$ and $S_1 + S_2 = totalSum$ eliminates S_2 : $2S_1 = totalSum + d$, so $S_1 = (totalSum + d)/2$. If $totalSum + d$ is odd, no integer S_1 satisfies this, so the answer is immediately 0. Otherwise, every way of choosing a subset summing to exactly this S_1 corresponds to exactly one valid partition (the chosen subset is S_1 , everything else is S_2 , and $S_2 = totalSum - S_1$ automatically satisfies $S_1 - S_2 = d$) – so the answer is exactly Section 1.4’s subset-count, evaluated at this derived target.

All Four Approaches

Every stage reuses Section 1.4’s implementation unchanged, called with $k = (totalSum + d)/2$. Only the space-optimized version, plus the reduction wrapper, is shown.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int countSubsetsSpaceOptimized(vector<int>& arr, int k) {
5      int n = arr.size();
6      vector<int> prev(k + 1, 0);
7
8      prev[0] = (arr[0] == 0) ? 2 : 1;
9      if (arr[0] != 0 && arr[0] <= k) prev[arr[0]] = 1;
10
11     for (int index = 1; index < n; index++) {
12         vector<int> current(k + 1, 0);
13         for (int target = 0; target <= k; target++) {
14             int notPick = prev[target];
15             int pick = (arr[index] <= target) ? prev[target - arr[index]] : 0;
16             current[target] = notPick + pick;
17         }
18         prev = current;
19     }
20     return prev[k];
21 }
22
23 int countPartitionsWithDiff(vector<int>& arr, int d) {
24     int totalSum = accumulate(arr.begin(), arr.end(), 0);
25     if ((totalSum + d) % 2 != 0 || totalSum + d < 0) return 0;
26
27     int s1 = (totalSum + d) / 2;
28     return countSubsetsSpaceOptimized(arr, s1);
29 }
30
31 int main() {
32     vector<int> arr = {1, 1, 2, 3};
33     cout << countPartitionsWithDiff(arr, 1) << endl; // 3

```

```

34     return 0;
35 }

```

Dry run

Dry Run. Take $arr = [1, 1, 2, 3]$, $d = 1$.

$totalSum = 7$. $S_1 = (7 + 1)/2 = 4$. Counting subsets summing to 4: $\{1, 3\}$ (using the first 1), $\{1, 3\}$ (using the second 1), and $\{1, 1, 2\}$ – three subsets, giving $answer = 3$. Checking one of them: subset $\{1, 3\}$ has sum 4; the complementary subset $\{1, 2\}$ has sum 3; difference $4 - 3 = 1 = d$, confirmed.

Complexity Analysis

Time Complexity. $O(n \cdot totalSum)$, inherited from Section 1.4. **Space Complexity.** $O(totalSum)$ for the rolling row.

Key Takeaway

Whenever a problem describes a partition in terms of a **sum and a difference** of the two parts, solving the resulting two-equation system for one part's target sum is almost always the fastest path to reusing an already-solved subset-counting DP, rather than inventing a new two-dimensional recurrence that tracks both parts directly.

1.6 A Note on Assign Cookies: When DP Is the Wrong Tool

Striver's sheet lists Assign Cookies again here, inside the DP on Subsequences lecture – and it is worth pausing on *why*, since the honest answer is a valuable lesson in its own right: **this problem does not need dynamic programming at all**, and forcing a DP solution onto it would be strictly worse than the greedy solution already derived in the Greedy chapter.

Recognising When a Subset/Assignment Problem Is Actually Greedy

Assign Cookies asks: given greed factors g and cookie sizes s , maximize the number of children satisfiable, where child i is satisfied by cookie j if $s[j] \geq g[i]$. This *looks* like an assignment problem that backtracking or DP might solve via some subset-matching recurrence – but the Greedy chapter's Assign Cookies section already proved, via a direct exchange argument, that sorting both arrays and greedily matching the smallest sufficient cookie to the smallest unsatisfied child is **provably optimal**. No DP table is needed because there is no genuine trade-off between overlapping sub-problems to resolve: the greedy choice at each step is never wrong, so there is nothing for memoization to usefully cache.

Why a DP/backtracking solution would be strictly worse here

A backtracking solution exploring every possible assignment of cookies to children would cost at least $O(2^n)$ in the worst case (each child either gets a cookie or doesn't, combinatorially), or, with a more clever subset-based DP state, perhaps $O(n \cdot m)$ or worse depending on formulation – but the Greedy chapter's greedy algorithm already solves the problem in $O(n \log n + m \log m)$ (dominated by sorting) with $O(1)$ extra space. Since the greedy algorithm is already proven optimal, any DP formulation can only ever *match* this bound at best, typically with strictly worse constants and memory usage – there is no scenario in which reaching for DP here is the better engineering decision.

The Real Lesson

Before reaching for memoization or tabulation on a subset/assignment-flavored problem, always ask first: *can a sorted greedy pass already solve this optimally?* The keyword-recognition habits from the Greedy chapter’s Assign Cookies section, in particular, should be checked *before* defaulting to a DP recurrence, not after. DP is the right tool precisely when greedy’s exchange argument *fails* – when the locally best choice can depend on choices made elsewhere, which is exactly what does *not* happen in Assign Cookies.

Key Takeaway

Not every problem that involves choosing a subset or making an assignment needs dynamic programming. Recognizing when a simpler, already-optimal greedy algorithm applies – and confidently *not* reaching for a DP table in that case – is as important a skill as knowing how to build the DP table when one is genuinely needed.

1.7 Target Sum

Problem Statement. Given an array *nums* and an integer *target*, assign either a + or – sign to each element, and count the number of distinct sign assignments that make the total expression evaluate to exactly *target*.

Pattern Recognition

“Assign + or – to each element to reach a target sum” is, word for word, Section 1.5’s partition problem in disguise: every element assigned + belongs to one “subset” *P* (positive sum), every element assigned – belongs to the other subset *N* (negative sum), and the expression’s value is exactly $P - N$. Counting sign assignments reaching *target* is therefore exactly counting partitions with difference $d = target$ – **no new derivation is needed at all.**

Understanding the reduction

Every element of *nums* goes into exactly one of two groups: those assigned + (summing to *P*) and those assigned – (summing to *N*, as a positive magnitude). The expression’s total value is $P - N$, and $P + N$ is always the array’s total sum (every element contributes its magnitude to exactly one of the two groups). This is precisely Section 1.5’s setup, with $d = target$: count subsets summing to $(totalSum + target)/2$.

All Four Approaches

Every stage is Section 1.5’s implementation, called unchanged with $d = target$.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int countSubsetsSpaceOptimized(vector<int>& arr, int k) {
5     int n = arr.size();
6     vector<int> prev(k + 1, 0);
7
8     prev[0] = (arr[0] == 0) ? 2 : 1;
9     if (arr[0] != 0 && arr[0] <= k) prev[arr[0]] = 1;
10
11     for (int index = 1; index < n; index++) {

```

```

12     vector<int> current(k + 1, 0);
13     for (int target = 0; target <= k; target++) {
14         int notPick = prev[target];
15         int pick = (arr[index] <= target) ? prev[target - arr[index]
16             ] : 0;
17         current[target] = notPick + pick;
18     }
19     prev = current;
20     return prev[k];
21 }
22
23 int findTargetSumWays(vector<int>& nums, int target) {
24     int totalSum = accumulate(nums.begin(), nums.end(), 0);
25     if ((totalSum + target) % 2 != 0 || totalSum + target < 0) return
26         0;
27     int p = (totalSum + target) / 2;
28     return countSubsetsSpaceOptimized(nums, p);
29 }
30
31 int main() {
32     vector<int> nums = {1, 1, 1, 1, 1};
33     cout << findTargetSumWays(nums, 3) << endl; // 5
34     return 0;
35 }

```

Dry run

Dry Run. Take $nums = [1, 1, 1, 1, 1]$, $target = 3$.

$totalSum = 5$. $P = (5 + 3)/2 = 4$. Counting subsets of five 1s summing to 4: there are $\binom{5}{4} = 5$ ways to choose which four of the five 1s are positive (the remaining one is negative), giving 5 valid sign assignments – each evaluates to $1 + 1 + 1 + 1 - 1 = 3 = target$, confirming the answer.

Complexity Analysis

Time Complexity. $O(n \cdot totalSum)$, inherited from Section 1.4. **Space Complexity.** $O(totalSum)$ for the rolling row.

Key Takeaway

This section is intentionally almost content-free beyond the reduction itself: once Count Partitions with a Given Difference is solved, Target Sum requires recognizing the disguise and nothing more. A surprising number of seemingly distinct DP problems on interview sheets are exactly this – the same two or three core recurrences, wearing different cover stories.

1.8 0/1 Knapsack

Problem Statement. Given n items, where item i has weight $wt[i]$ and value $val[i]$, and a knapsack of capacity W , choose a subset of items (each usable **at most once**) whose total weight does not exceed W , maximizing total value.

Pattern Recognition

This is the chapter's namesake, and it generalizes Section 1.2's boolean reachability table in the most direct way possible: instead of asking “*can* we reach exactly capacity *target*,” we ask “what is the *maximum value* achievable using *at most* capacity *W*” – replacing the logical OR combination with a **maximum**, and adding each picked item's *value* (not just consuming its weight) to the running total. Contrast this directly with the Greedy chapter's Fractional Knapsack problem: there, items could be split, which let a single greedy ratio-sort solve it optimally in $O(n \log n)$; here, items are all-or-nothing, which destroys that greedy argument and is exactly why this problem needs a genuine DP table.

Deriving the recurrence

Let $f(\text{index}, \text{capacity})$ be the maximum value achievable using items $0..index$, subject to a weight budget of capacity . At item $index$: either exclude it ($f(\text{index}-1, \text{capacity})$), or include it (only if $wt[index] \leq \text{capacity}$, gaining $val[index]$ plus $f(\text{index}-1, \text{capacity} - wt[index])$) – take whichever is larger:

$$f(\text{index}, \text{capacity}) = \max \left(f(\text{index}-1, \text{capacity}), \text{take} \right),$$

$$\text{take} = wt[index] \leq \text{capacity} ? val[index] + f(\text{index}-1, \text{capacity} - wt[index]) : -\infty.$$

Base case: $f(0, \text{capacity}) = val[0]$ if $wt[0] \leq \text{capacity}$, else 0.

Approach 1: Recursion

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int knapsackRecursive(int index, int capacity, vector<int>& wt, vector<
  int>& val) {
5     if (index == 0) {
6         return (wt[0] <= capacity) ? val[0] : 0;
7     }
8
9     int notPick = knapsackRecursive(index - 1, capacity, wt, val);
10    int pick = INT_MIN;
11    if (wt[index] <= capacity) {
12        pick = val[index] + knapsackRecursive(index - 1, capacity - wt[
          index], wt, val);
13    }
14    return max(notPick, pick);
15 }
16
17 int main() {
18     vector<int> wt = {1, 3, 4, 5};
19     vector<int> val = {1, 4, 5, 7};
20     int n = wt.size(), W = 7;
21     cout << knapsackRecursive(n - 1, W, wt, val) << endl; // 9
22     return 0;
23 }
```

Approach 1 Complexity

Time. $O(2^n)$. **Space.** $O(n)$ recursion stack.

Approach 2: Memoization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int knapsackMemo(int index, int capacity, vector<int>& wt, vector<int>& val,
5                 vector<vector<int>>& memo) {
6     if (index == 0) {
7         return (wt[0] <= capacity) ? val[0] : 0;
8     }
9     if (memo[index][capacity] != -1) return memo[index][capacity];
10
11     int notPick = knapsackMemo(index - 1, capacity, wt, val, memo);
12     int pick = INT_MIN;
13     if (wt[index] <= capacity) {
14         pick = val[index] + knapsackMemo(index - 1, capacity - wt[index], wt, val, memo);
15     }
16     return memo[index][capacity] = max(notPick, pick);
17 }
18
19 int main() {
20     vector<int> wt = {1, 3, 4, 5};
21     vector<int> val = {1, 4, 5, 7};
22     int n = wt.size(), W = 7;
23     vector<vector<int>> memo(n, vector<int>(W + 1, -1));
24     cout << knapsackMemo(n - 1, W, wt, val, memo) << endl; // 9
25     return 0;
26 }

```

Approach 2 Complexity

Time. $O(n \cdot W)$. **Space.** $O(n \cdot W)$ memo + $O(n)$ recursion stack.

Approach 3: Tabulation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int knapsackTabulation(vector<int>& wt, vector<int>& val, int W) {
5     int n = wt.size();
6     vector<vector<int>> dp(n, vector<int>(W + 1, 0));
7
8     for (int capacity = wt[0]; capacity <= W; capacity++) dp[0][capacity] = val[0];
9
10    for (int index = 1; index < n; index++) {
11        for (int capacity = 0; capacity <= W; capacity++) {
12            int notPick = dp[index - 1][capacity];
13            int pick = INT_MIN;
14            if (wt[index] <= capacity) {
15                pick = val[index] + dp[index - 1][capacity - wt[index]];
16            }
17            dp[index][capacity] = max(notPick, pick);
18        }
19    }
20 }

```

```

18     }
19 }
20 return dp[n - 1][W];
21 }
22
23 int main() {
24     vector<int> wt = {1, 3, 4, 5};
25     vector<int> val = {1, 4, 5, 7};
26     cout << knapsackTabulation(wt, val, 7) << endl; // 9
27     return 0;
28 }

```

Dry run of the tabulation table

Dry Run. Take $wt = [1, 3, 4, 5]$, $val = [1, 4, 5, 7]$, $W = 7$.

$index \backslash W$	0	1	2	3	4	5	6	7
0 (wt1, val1)	0	1	1	1	1	1	1	1
1 (wt3, val4)	0	1	1	4	5	5	5	5
2 (wt4, val5)	0	1	1	4	5	6	6	9
3 (wt5, val7)	0	1	1	4	5	7	8	9

At $index = 2$, $W = 7$: not-pick gives $dp[1][7] = 5$; pick gives $val[2] + dp[1][3] = 5 + 4 = 9$. Max is 9. The final answer is $dp[3][7] = 9$, achieved by items 1 (wt3, val4) and 2 (wt4, val5) together: total weight $3 + 4 = 7 \leq 7$, total value $4 + 5 = 9$, matching the table.

Approach 3 Complexity

Time. $O(n \cdot W)$. **Space.** $O(n \cdot W)$ table, no recursion stack.

Approach 4: Space Optimization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int knapsackSpaceOptimized(vector<int>& wt, vector<int>& val, int W) {
5     int n = wt.size();
6     vector<int> prev(W + 1, 0);
7
8     for (int capacity = wt[0]; capacity <= W; capacity++) prev[capacity] = val[0];
9
10    for (int index = 1; index < n; index++) {
11        vector<int> current(W + 1, 0);
12        for (int capacity = 0; capacity <= W; capacity++) {
13            int notPick = prev[capacity];
14            int pick = INT_MIN;
15            if (wt[index] <= capacity) {
16                pick = val[index] + prev[capacity - wt[index]];
17            }
18            current[capacity] = max(notPick, pick);
19        }
20        prev = current;
21    }
22    return prev[W];
23 }

```

```

24
25 int main() {
26     vector<int> wt = {1, 3, 4, 5};
27     vector<int> val = {1, 4, 5, 7};
28     cout << knapsackSpaceOptimized(wt, val, 7) << endl; // 9
29     return 0;
30 }

```

Approach 4 Complexity

Time. $O(n \cdot W)$. **Space.** $O(W)$ for the rolling row.

Key Takeaway

0/1 Knapsack is the most general member of this chapter's 0/1 family: Subset Sum is the special case where every item's weight equals its value (so reachability and value-maximization coincide), and Count Subsets is the special case that counts qualifying selections instead of optimizing a value. Recognizing 0/1 Knapsack as the "parent" recurrence of everything from Section 1.2 onward consolidates this entire half of the chapter into one shape.

1.9 Coin Change (Minimum Coins)

We now begin the chapter's **unbounded knapsack** family: every problem from here on lets each item (coin denomination, rod length) be used an **unlimited** number of times, which means the pick branch stays at the same index instead of advancing – exactly the recursion shape the Recursion chapter's Combination Sum section already derived.

Problem Statement. Given an array *coins* of denominations (each available in **unlimited** supply) and a target *amount*, find the **minimum** number of coins needed to make exactly *amount*, or -1 if it is impossible.

Pattern Recognition

"Unlimited supply of each denomination, minimize the count used" is the unbounded-knapsack analogue of Section 1.2's boolean reachability table: the pick branch stays at the same index (since the same coin may be reused), and the combination operator is a **minimum** of counts instead of a logical OR of reachability.

Deriving the recurrence

Let $f(\text{index}, \text{amount})$ be the minimum number of coins (using denominations $0..\text{index}$) needed to make *amount*. Either we exclude denomination *index* entirely ($f(\text{index} - 1, \text{amount})$), or we use it at least once (paying 1 coin, then recursing on the *same* index since it remains available, with the reduced amount):

$$f(\text{index}, \text{amount}) = \min \left(f(\text{index}-1, \text{amount}), \text{take} \right),$$

$$\text{take} = \text{coins}[\text{index}] \leq \text{amount} ? 1 + f(\text{index}, \text{amount} - \text{coins}[\text{index}]) : \infty.$$

Base case: $f(0, \text{amount}) = \text{amount}/\text{coins}[0]$ if *amount* is exactly divisible by *coins*[0], else ∞ (using only the first denomination, either it divides evenly or it is impossible).

Approach 1: Recursion

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const int INF = 1e9;
4
5 int coinChangeRecursive(int index, int amount, vector<int>& coins) {
6     if (index == 0) {
7         return (amount % coins[0] == 0) ? amount / coins[0] : INF;
8     }
9
10    int notTake = coinChangeRecursive(index - 1, amount, coins);
11    int take = INF;
12    if (coins[index] <= amount) {
13        take = 1 + coinChangeRecursive(index, amount - coins[index],
14                                       coins);
15    }
16    return min(notTake, take);
17 }
18
19 int main() {
20     vector<int> coins = {1, 2, 5};
21     int n = coins.size(), amount = 11;
22     int result = coinChangeRecursive(n - 1, amount, coins);
23     cout << (result >= INF ? -1 : result) << endl; // 3
24     return 0;
25 }

```

Approach 1 Complexity

Time. Exponential – each call can recurse on the same index many times (reducing the amount), giving a recursion tree that grows combinatorially with *amount*. **Space.** $O(\text{amount})$ recursion stack in the worst case (repeatedly taking the smallest coin).

Approach 2: Memoization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const int INF = 1e9;
4
5 int coinChangeMemo(int index, int amount, vector<int>& coins,
6                   vector<vector<int>>& memo) {
7     if (index == 0) {
8         return (amount % coins[0] == 0) ? amount / coins[0] : INF;
9     }
10    if (memo[index][amount] != -1) return memo[index][amount];
11
12    int notTake = coinChangeMemo(index - 1, amount, coins, memo);
13    int take = INF;
14    if (coins[index] <= amount) {
15        take = 1 + coinChangeMemo(index, amount - coins[index], coins,
16                                  memo);
17    }
18    return memo[index][amount] = min(notTake, take);
19 }

```

```

20 int main() {
21     vector<int> coins = {1, 2, 5};
22     int n = coins.size(), amount = 11;
23     vector<vector<int>> memo(n, vector<int>(amount + 1, -1));
24     int result = coinChangeMemo(n - 1, amount, coins, memo);
25     cout << (result >= INF ? -1 : result) << endl; // 3
26     return 0;
27 }

```

Approach 2 Complexity

Time. $O(n \cdot \text{amount})$. **Space.** $O(n \cdot \text{amount})$ memo + $O(\text{amount})$ recursion stack.

Approach 3: Tabulation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  const int INF = 1e9;
4
5  int coinChangeTabulation(vector<int>& coins, int amount) {
6      int n = coins.size();
7      vector<vector<int>> dp(n, vector<int>(amount + 1, INF));
8
9      for (int a = 0; a <= amount; a++) {
10         if (a % coins[0] == 0) dp[0][a] = a / coins[0];
11     }
12
13     for (int index = 1; index < n; index++) {
14         for (int a = 0; a <= amount; a++) {
15             int notTake = dp[index - 1][a];
16             int take = INF;
17             if (coins[index] <= a) {
18                 take = 1 + dp[index][a - coins[index]];
19             }
20             dp[index][a] = min(notTake, take);
21         }
22     }
23     int result = dp[n - 1][amount];
24     return result >= INF ? -1 : result;
25 }
26
27 int main() {
28     vector<int> coins = {1, 2, 5};
29     cout << coinChangeTabulation(coins, 11) << endl; // 3
30     return 0;
31 }

```

Dry run of the tabulation table

Dry Run. Take $\text{coins} = [1, 2, 5]$, $\text{amount} = 11$.

Row 0 (only coin 1): $dp[0][a] = a$ for all a (every amount needs exactly a coins of value 1).
 Row 1 (coins 1, 2): e.g. $dp[1][11] = \min(dp[0][11], 1 + dp[1][9])$; unrolling, using coin 2 as many times as possible gives $dp[1][11] = 6$ (5 twos and one... actually $11 = 2 \times 5 + 1$, so 5 coins of 2 plus 1 coin of 1 = 6 coins). Row 2 (coins 1, 2, 5): $dp[2][11] = \min(dp[1][11], 1 + dp[2][6])$; following this through, the optimal is $5 + 5 + 1 = 11$ using 3 coins, giving $dp[2][11] = 3$.

Approach 3 Complexity

Time. $O(n \cdot \text{amount})$. **Space.** $O(n \cdot \text{amount})$ table, no recursion stack.

Approach 4: Space Optimization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const int INF = 1e9;
4
5 int coinChangeSpaceOptimized(vector<int>& coins, int amount) {
6     int n = coins.size();
7     vector<int> prev(amount + 1, INF);
8
9     for (int a = 0; a <= amount; a++) {
10         if (a % coins[0] == 0) prev[a] = a / coins[0];
11     }
12
13     for (int index = 1; index < n; index++) {
14         vector<int> current(amount + 1, INF);
15         for (int a = 0; a <= amount; a++) {
16             int notTake = prev[a];
17             int take = INF;
18             if (coins[index] <= a) {
19                 take = 1 + current[a - coins[index]]; // same row! (
20                 // unbounded)
21             }
22             current[a] = min(notTake, take);
23         }
24         prev = current;
25     }
26     int result = prev[amount];
27     return result >= INF ? -1 : result;
28 }
29
30 int main() {
31     vector<int> coins = {1, 2, 5};
32     cout << coinChangeSpaceOptimized(coins, 11) << endl; // 3
33     return 0;
34 }
```

Why “Take” Reads from *current*, Not *prev*

Because the pick branch in an **unbounded** knapsack recurrence stays at the *same* index, the space-optimized version must read the take-branch value from the row currently being built (*current*), not from the previous row (*prev*) – the opposite of every 0/1 knapsack-family problem earlier in this chapter, where both branches read only from *prev*. This single detail is the only change space optimization needs when moving from the 0/1 family to the unbounded family.

Approach 4 Complexity

Time. $O(n \cdot \text{amount})$. **Space.** $O(\text{amount})$ for the rolling row.

Key Takeaway

Coin Change is 0/1 Knapsack's unbounded twin: same boolean-to-minimum combination idea, but with the pick branch staying at the same index – exactly the same 0/1-versus-unbounded distinction the Recursion chapter drew between Subset Sum II (no reuse) and Combination Sum (unlimited reuse). The single space-optimization wrinkle above (reading “take” from the current row) is the one new implementation detail this unbounded family introduces.

1.10 Coin Change II (Count the Ways)

Problem Statement. Given an array *coins* of denominations (each in unlimited supply) and a target *amount*, count the **number of distinct combinations** of coins that sum to exactly *amount*.

Pattern Recognition

This is Section 1.9's recurrence with minimum replaced by **addition** – the same exists-vs-count distinction drawn throughout this chapter and the Recursion chapter before it, now applied to the unbounded-reuse setting.

Deriving the recurrence

$$f(\text{index}, \text{amount}) = f(\text{index}-1, \text{amount}) + \text{take},$$

$$\text{take} = \text{coins}[\text{index}] \leq \text{amount} ? f(\text{index}, \text{amount} - \text{coins}[\text{index}]) : 0.$$

Base case: $f(0, \text{amount}) = 1$ if *amount* is exactly divisible by *coins*[0] (exactly one way: use that many copies of the first coin), else 0.

Approach 1: Recursion

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int changeRecursive(int index, int amount, vector<int>& coins) {
5     if (index == 0) {
6         return (amount % coins[0] == 0) ? 1 : 0;
7     }
8
9     int notTake = changeRecursive(index - 1, amount, coins);
10    int take = 0;
11    if (coins[index] <= amount) {
12        take = changeRecursive(index, amount - coins[index], coins);
13    }
14    return notTake + take;
15 }
16
17 int main() {
18     vector<int> coins = {1, 2, 5};
19     int n = coins.size();
20     cout << changeRecursive(n - 1, 5, coins) << endl; // 4
21     return 0;
22 }
```

Approach 1 Complexity

Time. Exponential, same overlapping-subproblem blowup as Section 1.9. **Space.** $O(\text{amount})$ recursion stack.

Approach 2: Memoization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int changeMemo(int index, int amount, vector<int>& coins,
5               vector<vector<int>>& memo) {
6     if (index == 0) {
7         return (amount % coins[0] == 0) ? 1 : 0;
8     }
9     if (memo[index][amount] != -1) return memo[index][amount];
10
11     int notTake = changeMemo(index - 1, amount, coins, memo);
12     int take = 0;
13     if (coins[index] <= amount) {
14         take = changeMemo(index, amount - coins[index], coins, memo);
15     }
16     return memo[index][amount] = notTake + take;
17 }
18
19 int main() {
20     vector<int> coins = {1, 2, 5};
21     int n = coins.size(), amount = 5;
22     vector<vector<int>> memo(n, vector<int>(amount + 1, -1));
23     cout << changeMemo(n - 1, amount, coins, memo) << endl; // 4
24     return 0;
25 }
```

Approach 2 Complexity

Time. $O(n \cdot \text{amount})$. **Space.** $O(n \cdot \text{amount})$ memo + $O(\text{amount})$ recursion stack.

Approach 3: Tabulation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int changeTabulation(vector<int>& coins, int amount) {
5     int n = coins.size();
6     vector<vector<long long>> dp(n, vector<long long>(amount + 1, 0));
7
8     for (int a = 0; a <= amount; a++) {
9         dp[0][a] = (a % coins[0] == 0) ? 1 : 0;
10    }
11
12    for (int index = 1; index < n; index++) {
13        for (int a = 0; a <= amount; a++) {
14            long long notTake = dp[index - 1][a];
15            long long take = (coins[index] <= a) ? dp[index][a - coins[
16                index]] : 0;

```

```

16         dp[index][a] = notTake + take;
17     }
18 }
19 return dp[n - 1][amount];
20 }
21
22 int main() {
23     vector<int> coins = {1, 2, 5};
24     cout << changeTabulation(coins, 5) << endl; // 4
25     return 0;
26 }

```

Dry run of the tabulation table

Dry Run. Take $coins = [1, 2, 5]$, $amount = 5$.

The four ways to make 5: $\{1, 1, 1, 1, 1\}$, $\{1, 1, 1, 2\}$, $\{1, 2, 2\}$, $\{5\}$ – matching $dp[2][5] = 4$. Row 0 (coin 1 only): $dp[0][a] = 1$ for every a (always exactly one way using only 1s). Row 1 (coins 1, 2): combinations using some 2s and the rest 1s – $dp[1][5] = 3$ (zero, one, or two 2s, with the remainder in 1s). Row 2 adds the option of using coin 5 once, contributing the fourth way.

Approach 3 Complexity

Time. $O(n \cdot amount)$. **Space.** $O(n \cdot amount)$ table, no recursion stack.

Approach 4: Space Optimization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int changeSpaceOptimized(vector<int>& coins, int amount) {
5     int n = coins.size();
6     vector<long long> prev(amount + 1, 0);
7
8     for (int a = 0; a <= amount; a++) {
9         prev[a] = (a % coins[0] == 0) ? 1 : 0;
10    }
11
12    for (int index = 1; index < n; index++) {
13        vector<long long> current(amount + 1, 0);
14        for (int a = 0; a <= amount; a++) {
15            long long notTake = prev[a];
16            long long take = (coins[index] <= a) ? current[a - coins[
17                index]] : 0;
18            current[a] = notTake + take;
19        }
20        prev = current;
21    }
22    return prev[amount];
23 }
24
25 int main() {
26     vector<int> coins = {1, 2, 5};
27     cout << changeSpaceOptimized(coins, 5) << endl; // 4
28     return 0;
29 }

```

Approach 4 Complexity

Time. $O(n \cdot \text{amount})$. **Space.** $O(\text{amount})$ for the rolling row, reading “take” from *current* exactly as in Section 1.9.

Key Takeaway

Coin Change and Coin Change II are, structurally, the exact same recurrence with minimum versus addition as the only difference – the unbounded-knapsack equivalent of Subset Sum versus Count Subsets earlier in this chapter. Once you have one of a min/count/exists triplet for a given recurrence shape, the other two cost almost nothing extra to derive.

1.11 Rod Cutting

Problem Statement. Given a rod of length n and an array *price* where *price*[i] is the price of a piece of length $i + 1$ (1-indexed), cut the rod into pieces (each possible length usable **any number of times**, since multiple pieces of the same length can come from different parts of the rod) to **maximize** total price.

Pattern Recognition

This closing problem of the chapter is a deliberate capstone connecting its two halves: Rod Cutting is **0/1 Knapsack’s value-maximization recurrence** (Section 1.8), but in the **unbounded-reuse** setting (Section 1.9’s pick-stays-at-the-same-index rule) – the “items” are piece lengths $1, \dots, n$, their “weights” are their own lengths, their “values” are their prices, and the “capacity” is the rod’s total length n .

Deriving the recurrence

Let $f(\text{index}, \text{length})$ be the maximum price obtainable from a rod of *length*, using piece lengths $1..index + 1$ (0-indexed *index* corresponds to piece length $index + 1$). Either exclude this piece length entirely ($f(\text{index} - 1, \text{length})$), or cut off one piece of this length (gaining *price*[*index*], then recursing on the *same* index since the same length remains available to cut again from what’s left):

$$f(\text{index}, \text{length}) = \max \left(f(\text{index}-1, \text{length}), \text{take} \right),$$

$$\text{take} = (\text{index}+1) \leq \text{length} ? \text{price}[\text{index}] + f(\text{index}, \text{length} - (\text{index}+1)) : -\infty.$$

Base case: $f(0, \text{length}) = \text{length} \times \text{price}[0]$ (using only length-1 pieces, cut exactly *length* of them).

Approach 1: Recursion

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int rodCuttingRecursive(int index, int length, vector<int>& price) {
5     if (index == 0) return length * price[0];
6
7     int notTake = rodCuttingRecursive(index - 1, length, price);
8     int take = INT_MIN;
9     int pieceLen = index + 1;
10    if (pieceLen <= length) {

```

```

11         take = price[index] + rodCuttingRecursive(index, length -
12             pieceLen, price);
13     }
14     return max(notTake, take);
15 }
16 int main() {
17     vector<int> price = {2, 5, 7, 8, 10};
18     int n = price.size();
19     cout << rodCuttingRecursive(n - 1, n, price) << endl; // 12
20     return 0;
21 }

```

Approach 1 Complexity

Time. Exponential – the same unbounded-reuse blowup as Coin Change. **Space.** $O(n)$ recursion stack.

Approach 2: Memoization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int rodCuttingMemo(int index, int length, vector<int>& price,
5     vector<vector<int>>& memo) {
6     if (index == 0) return length * price[0];
7     if (memo[index][length] != -1) return memo[index][length];
8
9     int notTake = rodCuttingMemo(index - 1, length, price, memo);
10    int take = INT_MIN;
11    int pieceLen = index + 1;
12    if (pieceLen <= length) {
13        take = price[index] + rodCuttingMemo(index, length - pieceLen,
14            price, memo);
15    }
16    return memo[index][length] = max(notTake, take);
17 }
18
19 int main() {
20     vector<int> price = {2, 5, 7, 8, 10};
21     int n = price.size();
22     vector<vector<int>> memo(n, vector<int>(n + 1, -1));
23     cout << rodCuttingMemo(n - 1, n, price, memo) << endl; // 12
24     return 0;
25 }

```

Approach 2 Complexity

Time. $O(n^2)$. **Space.** $O(n^2)$ memo + $O(n)$ recursion stack.

Approach 3: Tabulation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3

```

```

4  int rodCuttingTabulation(vector<int>& price) {
5      int n = price.size();
6      vector<vector<int>> dp(n, vector<int>(n + 1, 0));
7
8      for (int len = 0; len <= n; len++) dp[0][len] = len * price[0];
9
10     for (int index = 1; index < n; index++) {
11         for (int len = 0; len <= n; len++) {
12             int notTake = dp[index - 1][len];
13             int take = INT_MIN;
14             int pieceLen = index + 1;
15             if (pieceLen <= len) {
16                 take = price[index] + dp[index][len - pieceLen];
17             }
18             dp[index][len] = max(notTake, take);
19         }
20     }
21     return dp[n - 1][n];
22 }
23
24 int main() {
25     vector<int> price = {2, 5, 7, 8, 10};
26     cout << rodCuttingTabulation(price) << endl; // 12
27     return 0;
28 }

```

Dry run of the tabulation table

Dry Run. Take $price = [2, 5, 7, 8, 10]$ (rod length $n = 5$).

The optimal cut for a rod of length 5 with these prices is two pieces of length 2 (price 5 each) and one of length 1 (price 2): $5 + 5 + 2 = 12$. Tracing the table: row 0 (only length-1 pieces) gives $dp[0][5] = 5 \times 2 = 10$. Row 1 (lengths 1, 2) considers mixing in length-2 pieces: $dp[1][5] = \max(dp[0][5], price[1] + dp[1][3]) = \max(10, 5 + dp[1][3])$; unrolling $dp[1][3] = \max(dp[0][3], 5 + dp[1][1]) = \max(6, 5 + 2) = 7$, giving $dp[1][5] = \max(10, 5 + 7) = 12$. Later rows (introducing longer, less efficient-per-length pieces) do not improve on this, so $dp[4][5] = 12$.

Approach 3 Complexity

Time. $O(n^2)$. **Space.** $O(n^2)$ table, no recursion stack.

Approach 4: Space Optimization

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int rodCuttingSpaceOptimized(vector<int>& price) {
5      int n = price.size();
6      vector<int> prev(n + 1, 0);
7
8      for (int len = 0; len <= n; len++) prev[len] = len * price[0];
9
10     for (int index = 1; index < n; index++) {
11         vector<int> current(n + 1, 0);
12         for (int len = 0; len <= n; len++) {
13             int notTake = prev[len];

```

```

14         int take = INT_MIN;
15         int pieceLen = index + 1;
16         if (pieceLen <= len) {
17             take = price[index] + current[len - pieceLen]; // same
18             row
19         }
20         current[len] = max(notTake, take);
21     }
22     prev = current;
23 }
24 return prev[n];
25 }
26
27 int main() {
28     vector<int> price = {2, 5, 7, 8, 10};
29     cout << rodCuttingSpaceOptimized(price) << endl; // 12
30     return 0;
31 }

```

Approach 4 Complexity

Time. $O(n^2)$. **Space.** $O(n)$ for the rolling row.

Key Takeaway

Rod Cutting closes this chapter by making its organizing idea explicit one final time: every problem here is the same (*index, target*) table, varying along exactly two independent axes – *what the combination operator is* (OR, sum, or max/min) and *whether the pick branch reuses the current index or advances past it*. Four operators (OR, sum, min, max) times two reuse rules (0/1, unbounded) accounts for essentially every problem in this chapter, Rod Cutting included.